Database Fundamentals (6CS546)

Pandemic Resilience System (PRS)

Database Design and Development

Conceptual ER model, relational logical model (MS SQL Server) and document logical model (MongoDB)

Student name: [Your full name]

Student ID: [Your student ID]

Module: 6CS546 Database Fundamentals

Date: October 2026

Contents

1 Introduction
  1.1 Scenario summary
  1.2 Design approach
  1.3 Requirements
2 Conceptual Design
  2.1 Conceptual ER model (Figure 1, detail views 1a to 1c)
  2.2 Assumptions and design decisions
  2.3 Validating the model against the scenario
3 Logical Design
  3.1 Choosing a store for each part of the data
  3.2 Relational model (Figure 2, detail views 2a to 2c)
  3.3 Document model (Figure 3)
  3.4 Integrating the two models
4 Conclusion
References
Appendix A Implementation files and test evidence

1 Introduction

1.1 Scenario summary

The Pandemic Resilience System (PRS) is a government platform for managing critical activities during a pandemic. It serves three groups of users. Government officials set purchase limits and day-of-week purchase schedules for critical items, enrol and verify merchants, and validate vaccination records. Merchants, which operate one or more stores, check each customer's eligibility before a sale, record every purchase of a restricted item and keep their stock levels up to date. Members of the public enrol themselves and their children under 18, locate stores with stock, see whether they may buy today, and upload vaccination evidence either as a PDF or as a machine-readable FHIR report such as the NHS Immunization sample.

Every individual receives a PRS-Id that maps to one or more national identifiers (passport, driving licence, BRP). National identifiers and dates of birth are encrypted, only specific roles may decrypt them, and every access to sensitive data is audited.

1.2 Design approach

The design was produced in four steps:

- **Requirements.** The scenario was broken into numbered requirements (R1 to R18, Table 1) so coverage could be checked.
- **Conceptual model.** One technology-independent ER model (Figure 1) captures every entity, attribute and relationship, with sensitive attributes marked.
- **Classification.** Each part of the model was assigned to the store that suits it best (Table 2): structured, transactional data with strict integrity goes to MS SQL Server; semi-structured, unstructured, append-only or highly variable data goes to MongoDB. This is a polyglot persistence design.
- **Derivation.** Both logical models (Figures 2 and 3) were derived from the conceptual model using identical names for the same things, and linked through shared identifiers (PRSId, UserId, StoreId, OfficialId, UploadId). Both were then implemented and tested (Appendix A) to prove the models work, including importing the NHS FHIR sample.

All diagrams, the SQL Server DDL and the report tables are generated from one model definition, so a name cannot differ between the ERD, the schema and the collections.

1.3 Requirements

Id	Requirement (from the scenario)	Entities that satisfy it
R1	Assign every individual a unique PRS-Id	Person
R2	Map the PRS-Id to one or more national identifiers of different types	NationalIdentifier, IdentifierType
R3	Encrypt national identifiers and DOB; only authorised roles decrypt	[E] attributes; Permission (DECRYPT_*)
R4	Enrol family members (children under 18), each with their own PRS-Id	FamilyLink
R5	Purchase limits per item and period (e.g. 1 milk/day, 7	PurchaseLimit

	masks/week)	
R6	Purchase schedules by DOB (birth-year digit to day of week)	PurchaseSchedule, Person.BirthYearDigit
R7	Enrol merchants: licence number, name, operational addresses	Merchant, Store
R8	Separate account for each operational location	Store, UserAccount
R9	Record every purchase of critical items	Purchase, PurchaseLine
R10	Maintain stock levels; locate the nearest store with stock	StockLevel, Store.GeoLocation
R11	Check eligibility from DOB and purchase history before a sale	PurchaseSchedule, PurchaseLimit, Purchase
R12	Alert officials when individuals or businesses break limits	Alert
R13	Track vaccinations of residents and visitors; compliance per dose and period	VaccinationRecord, Vaccine, VaccinationPolicy
R14	Upload records as PDF or FHIR JSON/XML; save machine-readable reports automatically	VaccinationUpload, GridFS bucket PRSFiles
R15	Officials validate and verify records; use them at border checks	Verification, BorderCheck
R16	Role-based access control	UserAccount, Role, Permission
R17	Audit access to sensitive data	AuditLog
R18	Notify people of nearby stock; publish real-time news and vaccination schedules	Notification, Announcement

Table 1. Requirements and the entities that cover them.

2 Conceptual Design

2.1 Conceptual ER model

Figure 1 is the complete conceptual ER model in Crow's Foot notation. It is technology-independent: it shows entities, attributes, identifiers and relationships with cardinality and participation, but no data types, tables or foreign keys. Entities are coloured by functional area. Attributes marked [E] must be stored encrypted and attributes marked [S] are sensitive personal data. Because the full model has 26 entities, Figures 1a to 1c repeat it as three detail views at a readable size; entities belonging to another view appear there as dashed grey boxes. Full-resolution SVG files of every figure are supplied with this report.

Associative entities are used where a many-to-many relationship carries its own data: FamilyLink (guardian to dependant, with the relationship type), StockLevel (store to item, with quantity) and PurchaseLine (purchase to item, with quantity). The two M:N relationships that carry no data, UserAccount to Role and Role to Permission, are drawn as M:N and resolved in the logical model.

Figure 1. PRS conceptual ER model (Crow's Foot notation)

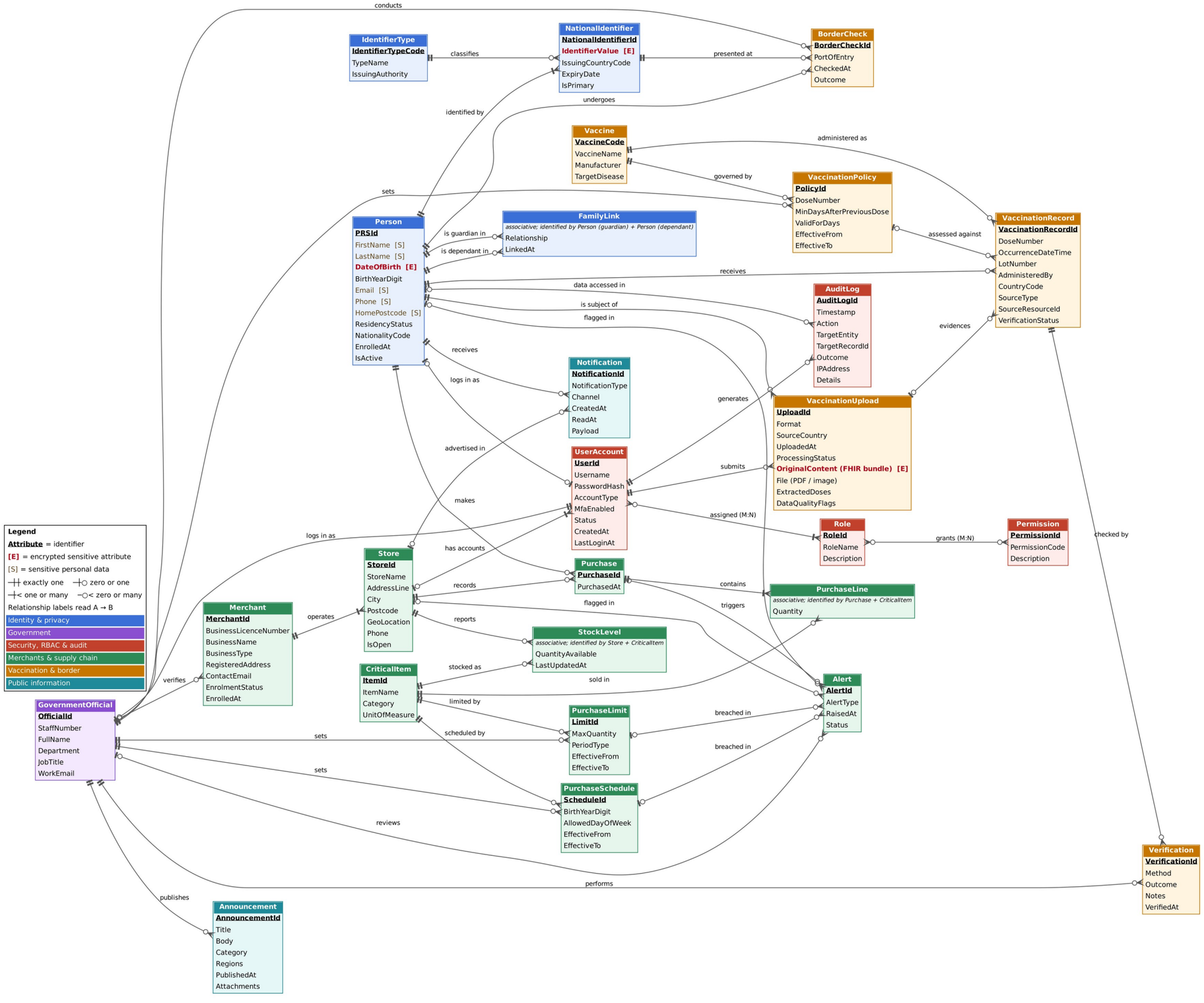


Figure 1. PRS conceptual ER model (complete). Full-resolution file: fig1_conceptual_erd.svg

Figure 1a. Conceptual ER model, detail view: people, identity, access and public information

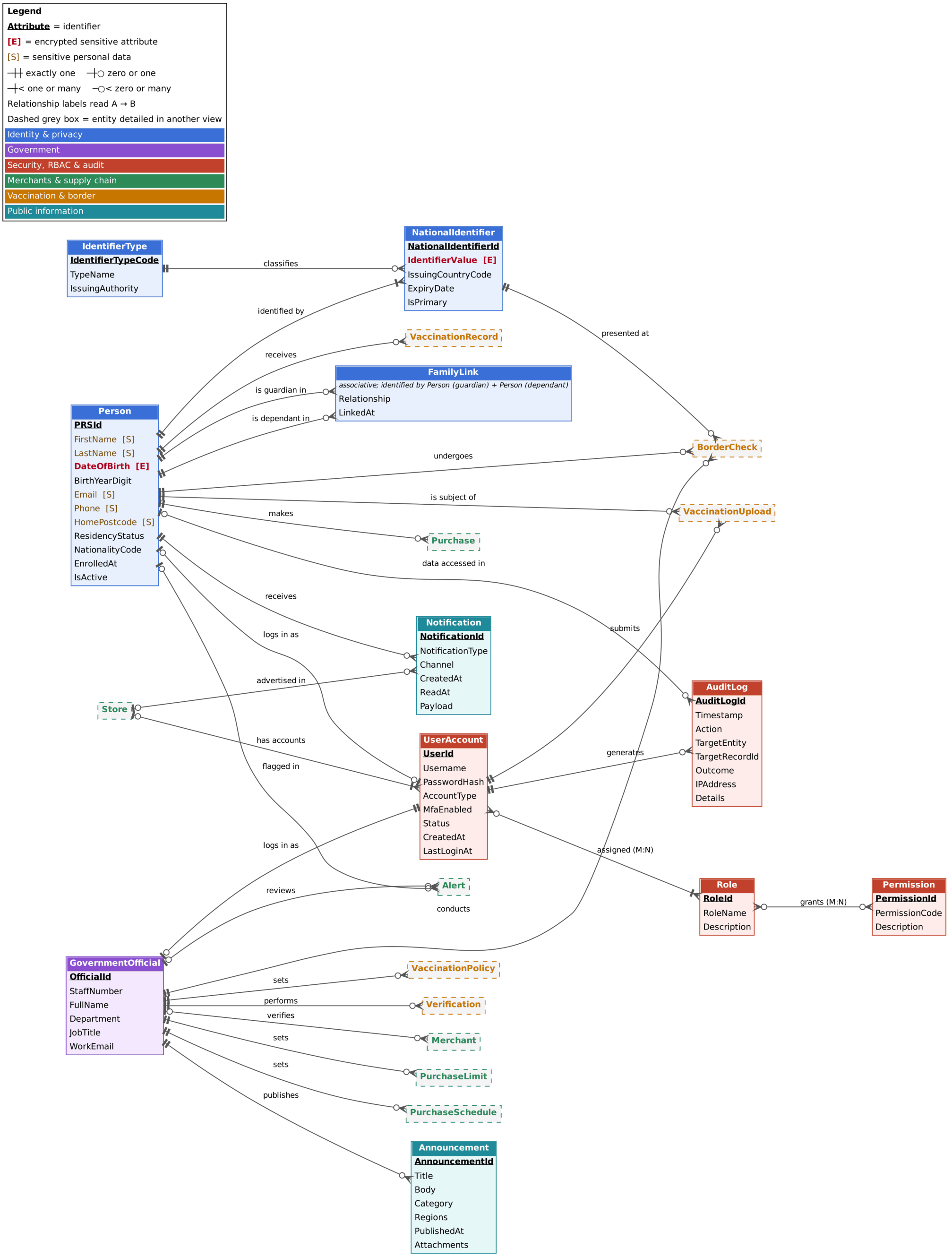


Figure 1a. Detail view: people, identity, access and public information.

Figure 1b. Conceptual ER model, detail view: merchants and supply chain

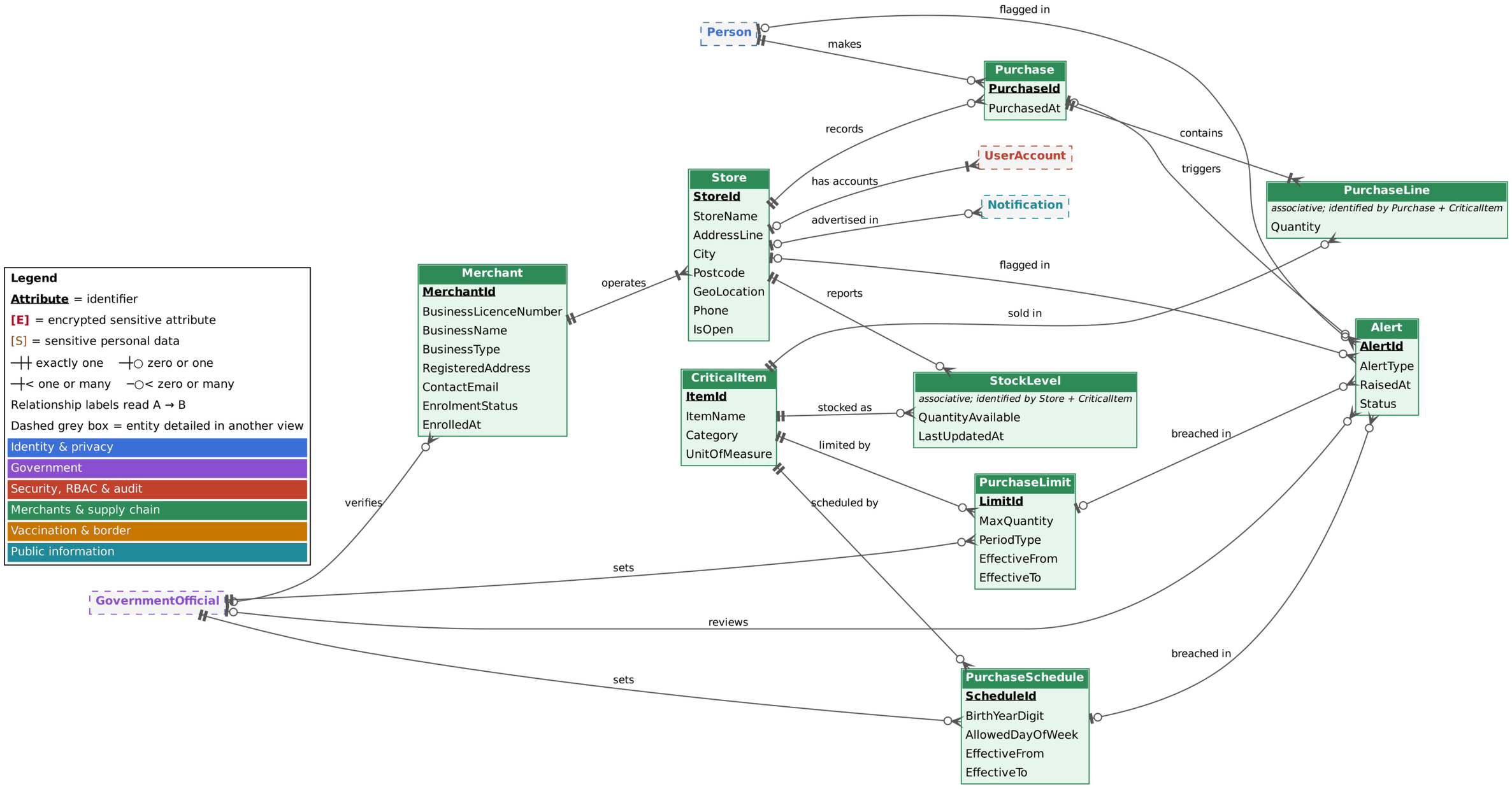


Figure 1b. Detail view: merchants and supply chain.

Figure 1c. Conceptual ER model, detail view: vaccination and border control

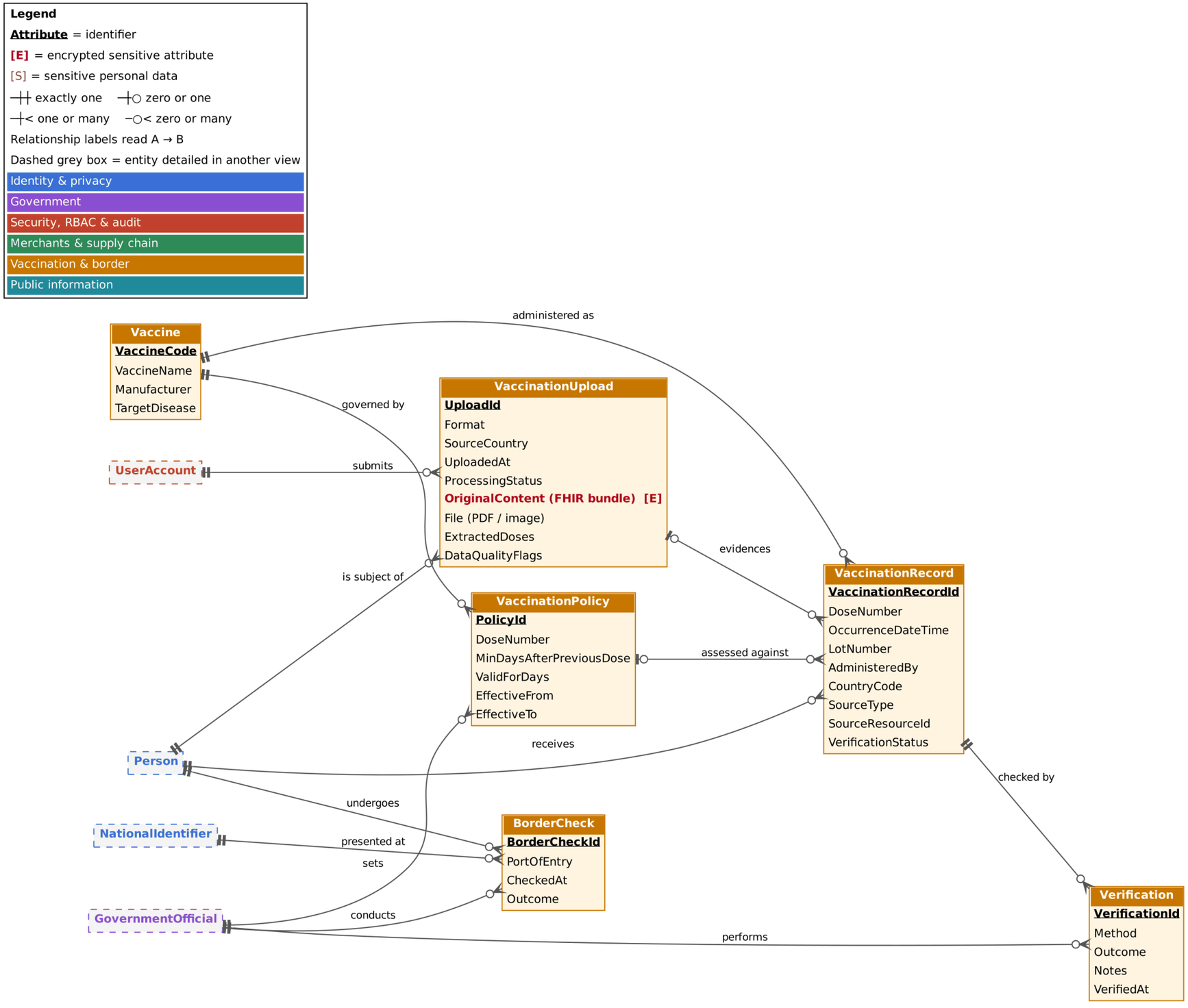


Figure 1c. Detail view: vaccination and border control.

2.2 Assumptions and design decisions

- **Identity.** A person can hold several identifiers (for example a passport and a BRP), so Person to NationalIdentifier is 1 to 1..N. IdentifierType is a separate entity so new identifier types can be added without changing the structure. One identifier can be flagged as primary.
- **Family members.** A child can have two guardians and a guardian can enrol several children, so FamilyLink resolves an M:N recursive relationship on Person. Children may have no UserAccount of their own; the guardian acts for them.
- **Schedules without decrypting DOB.** DateOfBirth is encrypted, but schedules depend only on the last digit of the birth year. BirthYearDigit is derived once at enrolment and stored in clear, so a store can check the schedule without decrypting the DOB. It reveals one digit only and is a deliberate, documented redundancy. The under-18 rule for dependants is checked by the enrolment service, which is allowed to decrypt the DOB.
- **Limits.** A purchase limit applies to a person across all stores, so eligibility uses the person's whole purchase history. A week runs Monday to Sunday.
- **Merchants and accounts.** A business (Merchant) operates one or more locations (Store). Each store has one or more UserAccounts. A UserAccount belongs to exactly one of a Person, a Store or a GovernmentOfficial (an exclusive arc).
- **Alerts.** An Alert concerns a person and/or a store, may be triggered by a purchase, and references the limit or schedule that was broken, so officials can see exactly which rule failed.
- **Vaccination evidence.** VaccinationUpload is the evidence a person submits (FHIR or PDF). A machine-readable upload yields one VaccinationRecord per dose; a PDF yields none until an official reads it and keys the doses in. Each record is assessed against the VaccinationPolicy for its vaccine and dose, and every check by an official is kept as a Verification, so the history of who verified what is preserved.
- **Vaccine codes.** Vaccine is identified by its SNOMED CT product code, the same code FHIR uses in vaccineCode, so imported reports link to it without translation.
- **Audit and information.** AuditLog records who did what to which record and when; Notification and Announcement support the public dashboard's real-time information.

2.3 Validating the model against the scenario

Each story below was traced through Figure 1 and then run against the implemented databases (Appendix A).

- **A pharmacy sells face masks.** The sample traveller was born in 1965 (BirthYearDigit 5). PurchaseSchedule allows digits 5 and 7 to buy face masks on Thursday, so a Monday attempt is refused and an Alert of type ScheduleBreach is raised. On Thursday five masks are sold (Purchase and PurchaseLine, StockLevel reduced). A further three that week would exceed PurchaseLimit (7 per week, summed from Purchase history), so they are refused with a LimitBreach alert.
- **A parent uploads a child's PDF certificate.** The parent's Person is linked to the child's Person through FamilyLink. The parent's UserAccount submits a VaccinationUpload whose PRSId is the child's; the PDF is stored as a file. An official

reviews it, creates VaccinationRecords with SourceType PDF and records a Verification.

- **A traveller uploads the NHS FHIR report.** The Patient's NHS number is matched to NationalIdentifier (type NHS_NUMBER) to find the PRS-Id and the birth date is compared with Person.DateOfBirth. The two Immunization resources become two VaccinationRecords linked to the upload, each assessed against VaccinationPolicy for its dose.
- **A border check.** The officer finds the person through the NationalIdentifier presented, checks the compliance of their verified VaccinationRecords and records a BorderCheck with its outcome.
- **Finding a mask nearby.** StockLevel joined to Store (with its location) gives the nearest stores with stock; PurchaseSchedule and PurchaseLimit tell the person whether they may buy today; a Notification can tell them proactively.

3 Logical Design

3.1 Choosing a store for each part of the data

Entities	Store	Reason
Person, NationalIdentifier, IdentifierType, FamilyLink	SQL Server	Core identity: must be unique, consistent and joined to almost everything; needs column encryption.
UserAccount, Role, Permission (+ UserRole, RolePermission)	SQL Server	Security rules need strict referential integrity; M:N is natural as junction tables.
Merchant, Store, CriticalItem, StockLevel	SQL Server	Structured master data; stock updates must be transactional; spatial index finds the nearest store.
PurchaseLimit, PurchaseSchedule, Purchase, PurchaseLine, Alert	SQL Server	Eligibility check, sale and stock change must happen in one ACID transaction; aggregate queries over history.
Vaccine, VaccinationPolicy, VaccinationRecord, Verification, BorderCheck	SQL Server	Normalised dose facts are queried at border checks and for compliance; must reference Person reliably.
VaccinationUpload (+ GridFS bucket PRSFiles)	MongoDB	FHIR bundles are nested and differ between countries; PDFs and images are unstructured binary files.
AuditLog	MongoDB	Very high write volume, append-only, details vary by action; time-series collection scales horizontally.
Notification, Announcement	MongoDB	Polymorphic payloads and attachments; short-lived, read-heavy feeds for the public dashboard.

Table 2. Data classification: relational versus document.

Structured data with strict integrity needs, transactions and many joins belongs in the relational model. Semi-structured data (FHIR), unstructured files (PDF and images) and high-volume, variable event data (audit, notifications) belong in the document model. Every conceptual entity appears in exactly one logical model, so no data is stored twice. The one exception is ExtractedDoses inside VaccinationUpload, which is a read-only snapshot of what was parsed from the source, kept as evidence.

3.2 Relational logical model (MS SQL Server)

Figure 2 shows all 24 tables with their columns, SQL Server data types, primary keys (PK), foreign keys (FK), unique constraints (UQ), NOT NULL (NN) and Always Encrypted columns. Figures 2a to 2c are readable detail views. The complete DDL, including CHECK constraints, indexes and stored procedures, is in prs_sqlserver_schema.sql (Appendix A).

Figure 2. PRS relational logical model (MS SQL Server)

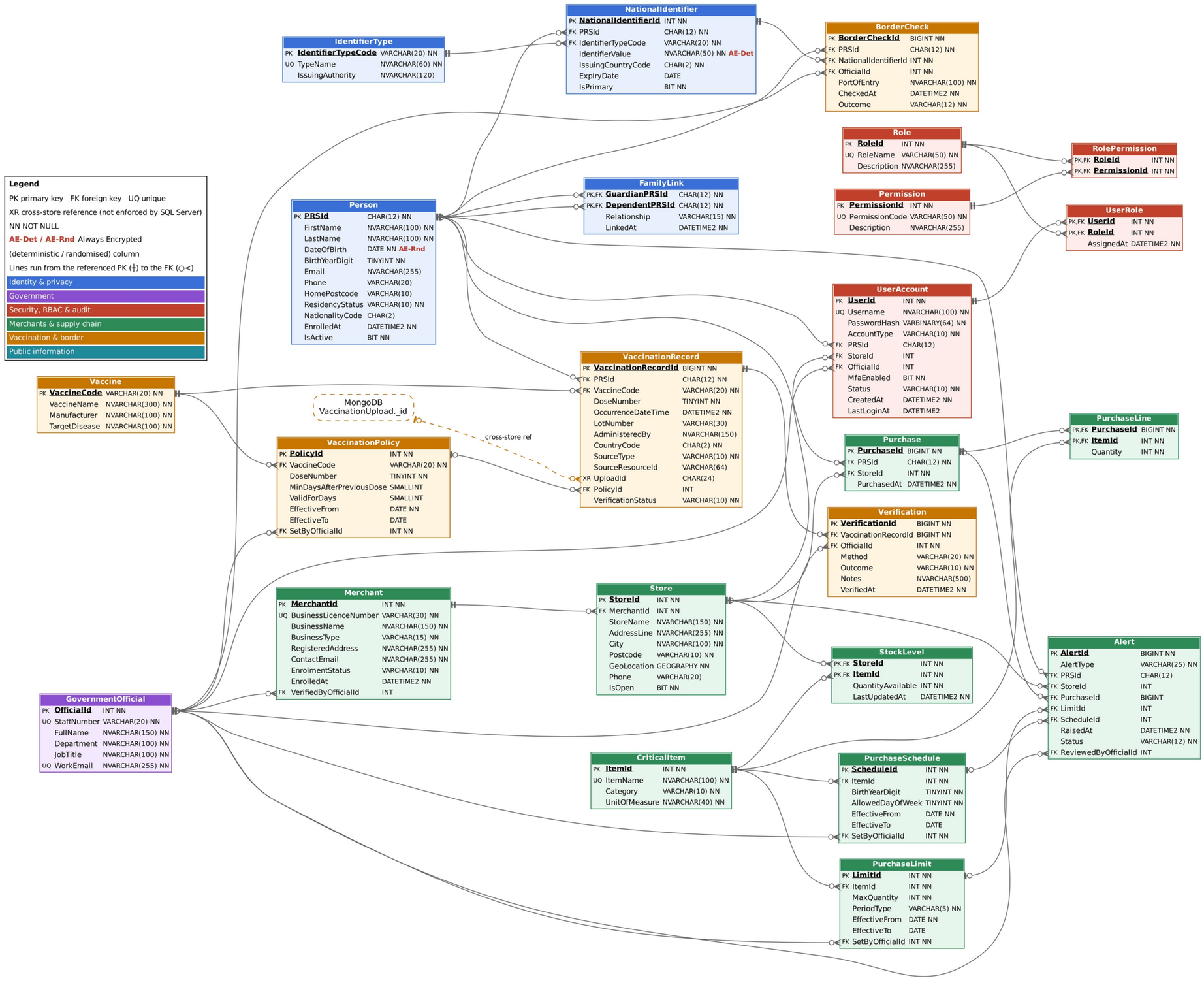


Figure 2. PRS relational logical model (complete). Full-resolution file: fig2_relational_schema.svg

Figure 2a. Relational model, detail view: people, identity, access and public information

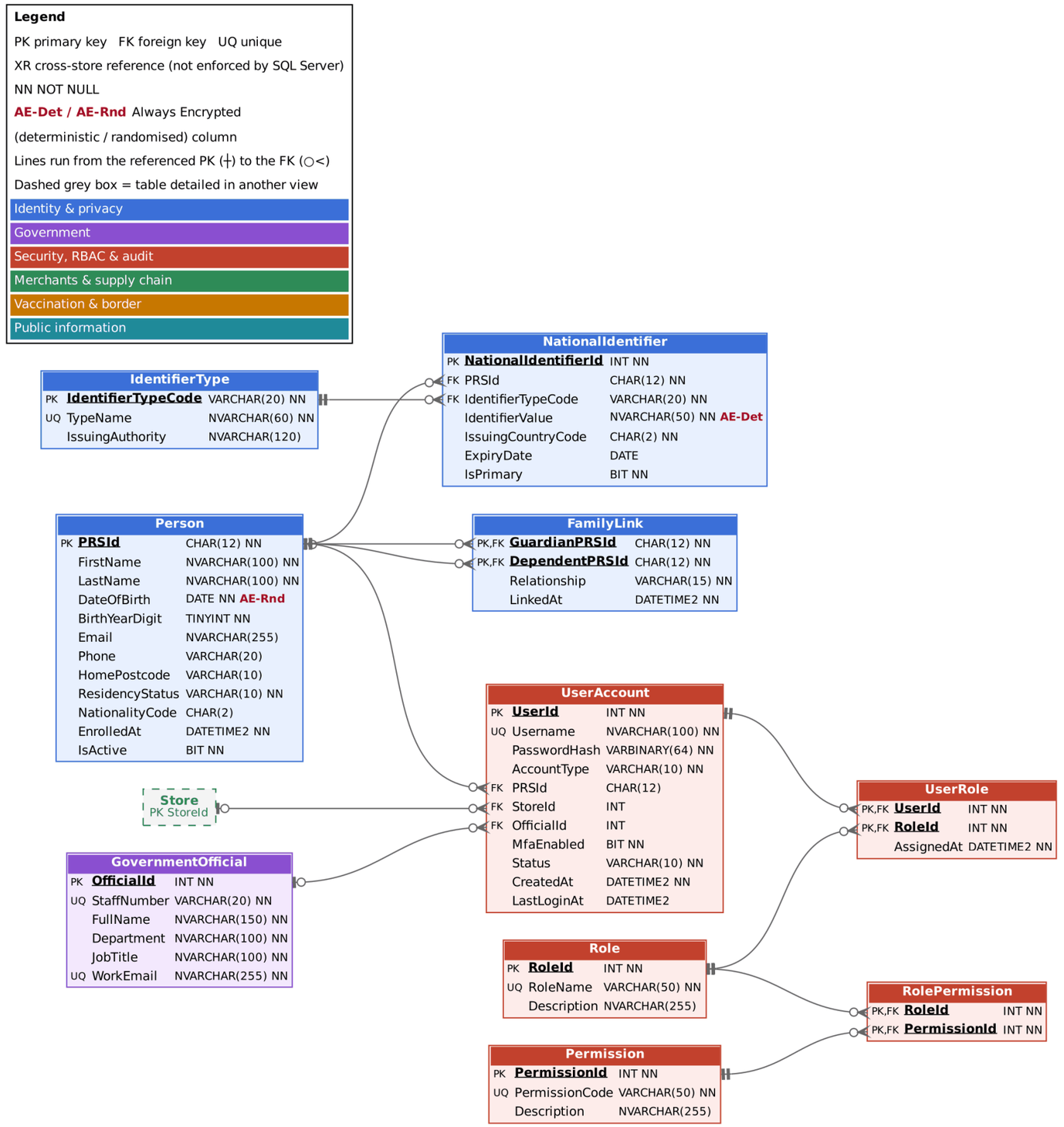


Figure 2a. Detail view: people, identity and access.

Figure 2b. Relational model, detail view: merchants and supply chain

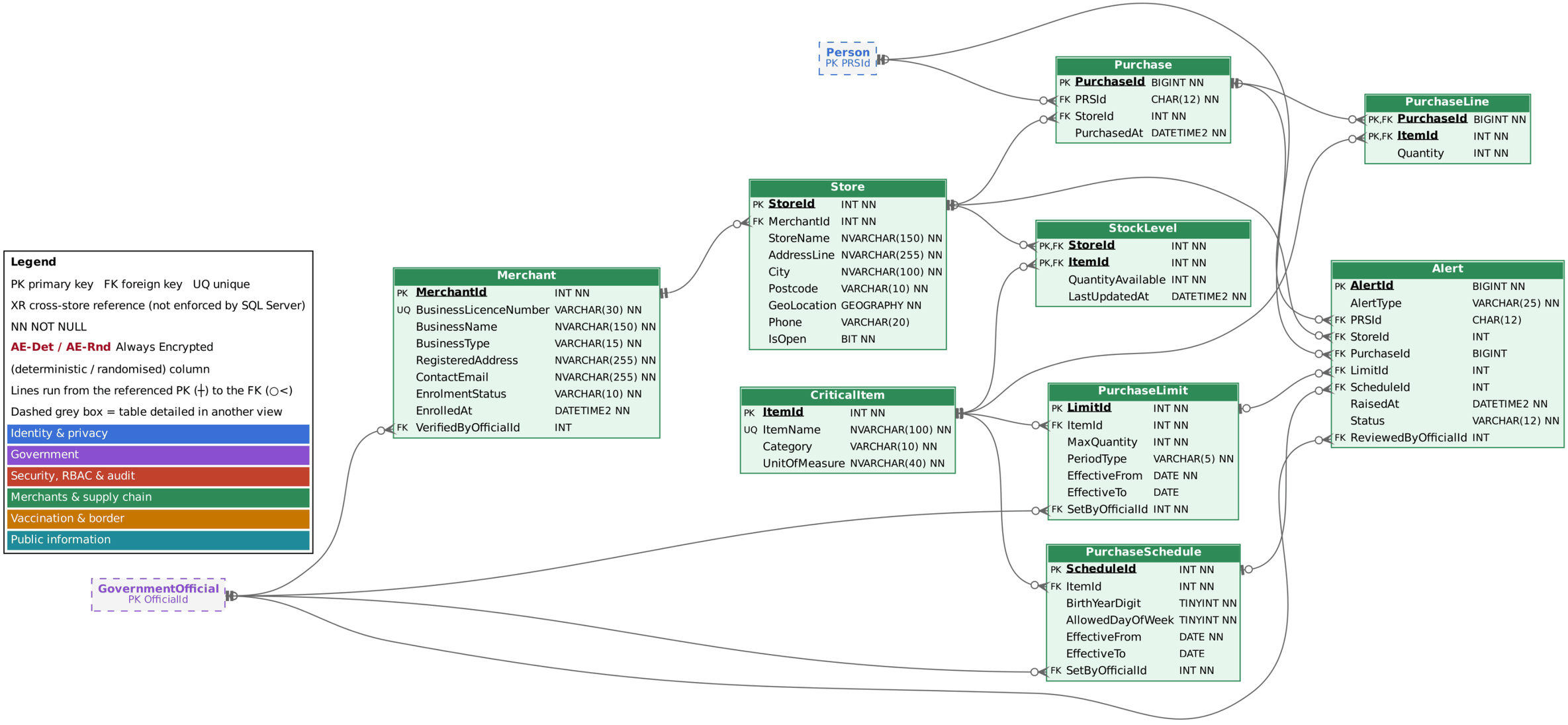


Figure 2b. Detail view: merchants and supply chain.

Figure 2c. Relational model, detail view: vaccination and border control

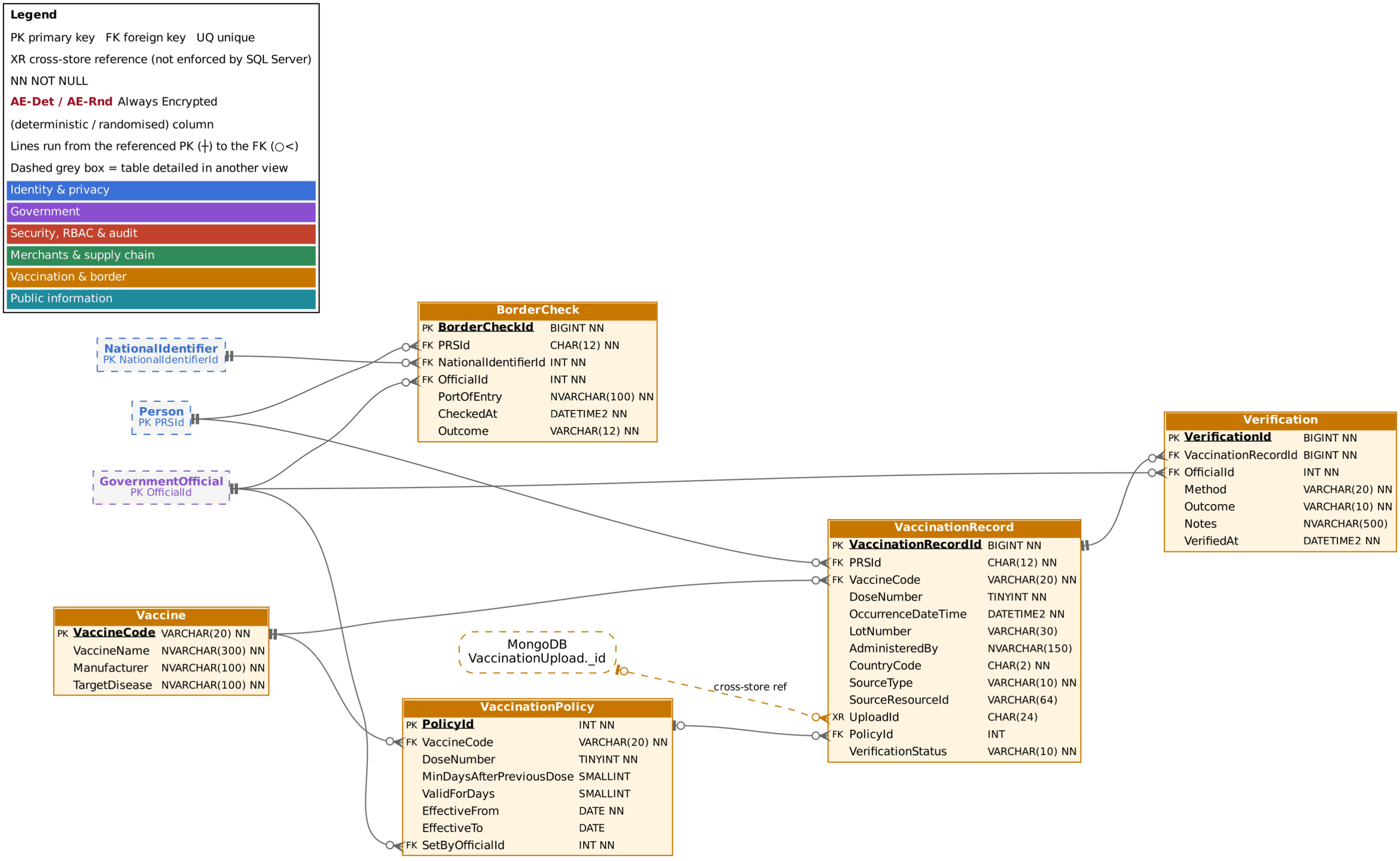


Figure 2c. Detail view: vaccination and border control.

Mapping from the conceptual model

- **Entities.** Each entity became a table with the same name, and its identifier became the primary key. PRSId keeps its business format (PRS + 9 digits, generated from a SEQUENCE). Other entities use IDENTITY surrogate keys, with natural keys such as BusinessLicenceNumber, StaffNumber and PermissionCode protected by UNIQUE constraints. Vaccine keeps its natural SNOMED code as the key because FHIR uses it directly.
- **1:N relationships.** Each 1:N relationship became a foreign key on the N side. Optional participation became a nullable FK (for example Alert.LimitId), mandatory participation a NOT NULL FK.
- **M:N relationships.** UserAccount to Role and Role to Permission became the junction tables UserRole and RolePermission. The associative entities became tables with composite primary keys: FamilyLink (GuardianPRSId, DependentPRSId), StockLevel (StoreId, ItemId) and PurchaseLine (PurchaseId, ItemId).
- **Exclusive arc.** The exclusive arc on UserAccount is enforced by a CHECK constraint: a Public account must have PRSId only, a Merchant account StoreId only, an Official account OfficialId only.

Normalisation

All tables are in third normal form. There are no repeating groups: a purchase's items are rows in PurchaseLine, a schedule has one row per birth-year digit rather than a list such as '0,2', and a person's identifiers are rows in NationalIdentifier. There are no partial dependencies: in StockLevel and PurchaseLine the quantity depends on the whole composite key. There are no transitive dependencies: vaccine name and manufacturer depend on VaccineCode, so they live in Vaccine, not in VaccinationRecord, and identifier type names live in IdentifierType. Two controlled redundancies are kept on purpose: Person.BirthYearDigit (section 2.2) and VaccinationRecord.VerificationStatus, the current outcome of the latest Verification, which keeps border checks to a single-table lookup.

Integrity and business rules

- **Domain rules.** CHECK constraints restrict codes and ranges (PeriodType in Day/Week, BirthYearDigit 0 to 9, AllowedDayOfWeek 1 to 7, QuantityAvailable >= 0, EffectiveTo >= EffectiveFrom, GuardianPRSId <> DependentPRSId).
- **Uniqueness.** UNIQUE (IdentifierTypeCode, IssuingCountryCode, IdentifierValue) stops the same passport being mapped to two people. A filtered unique index on (PRSId, SourceResourceId) makes FHIR imports idempotent: uploading the same report twice cannot create duplicate doses.
- **Procedures.** usp_CheckPurchaseEligibility evaluates the schedule and the limit against purchase history; usp_RecordPurchase runs the check, the sale and the stock decrement in one transaction and raises an Alert when a rule is broken; usp_FindNearestStock uses the spatial index on Store.GeoLocation; vw_VaccinationCompliance reports compliance per dose.

Security in the relational model

- **Encryption ([E], AE-Det/AE-Rnd).** NationalIdentifier.IdentifierValue uses deterministic Always Encrypted, so equality lookups still work (border checks, FHIR identity matching, the uniqueness constraint) while the server never sees plaintext. Person.DateOfBirth uses randomised encryption, which is stronger and never needs to be searched. Keys are held outside the database (e.g. Azure Key Vault) and released only to services acting for a role with DECRYPT_NATIONAL_ID or DECRYPT_DOB.
- **Sensitive data ([S]).** Other personal data (names, contact details, postcodes) is protected by Transparent Data Encryption at rest, Dynamic Data Masking for roles that do not need it, and TLS in transit.
- **RBAC.** Roles and permissions are data (Role, Permission, RolePermission), so officials can change who may do what without code changes. A row-level security predicate based on FamilyLink lets a Citizen see only their own and their dependants' rows.

3.3 Document logical model (MongoDB)

Figure 3 shows the four collections and the GridFS bucket, their fields and types, which parts are embedded (italic) and how each collection references SQL Server through shared keys (dashed). The validators, indexes and sample data are in prs_mongodb_setup.js.

Figure 3. PRS document logical model (MongoDB)

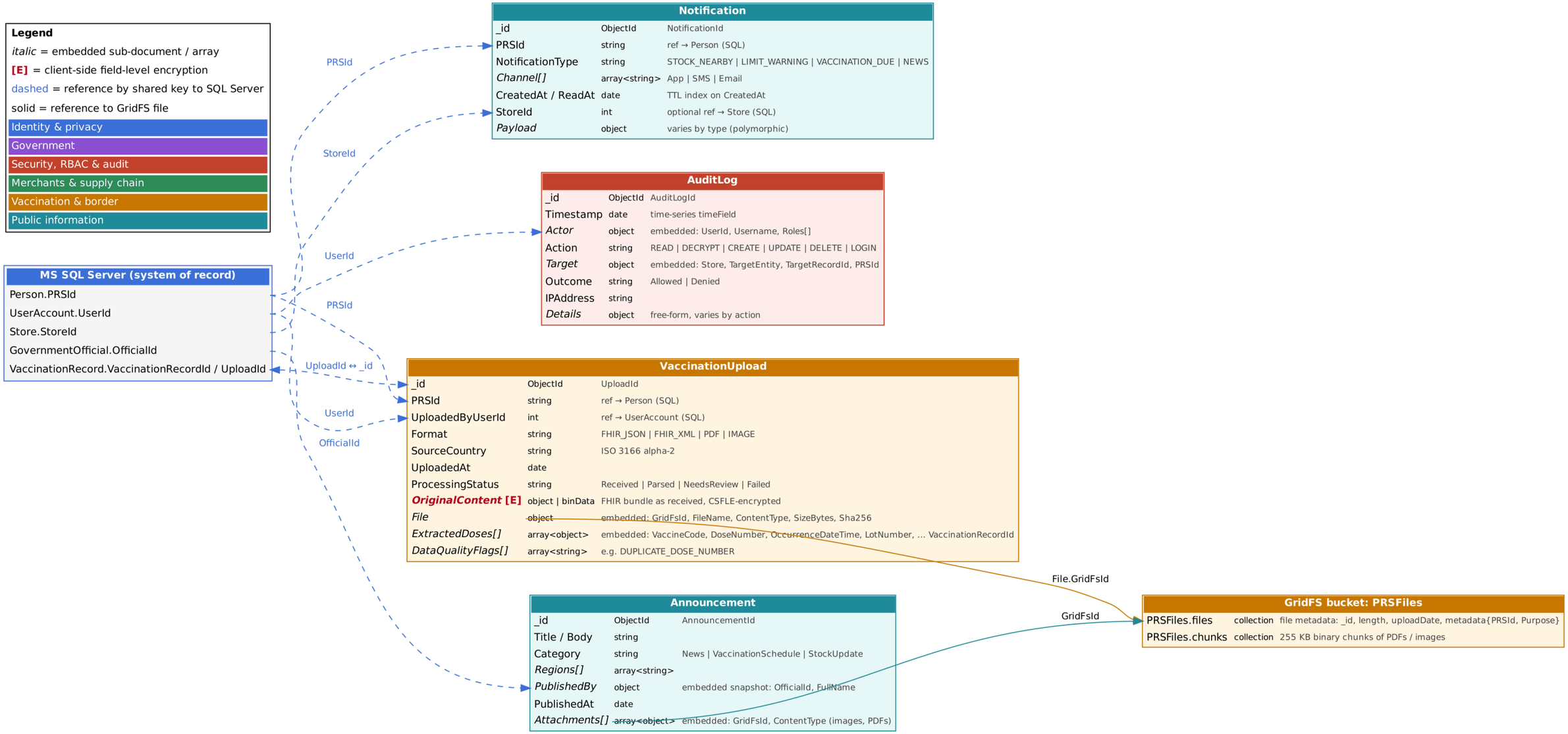


Figure 3. PRS document logical model. Full-resolution file: fig3_document_model.svg

Collections

- **VaccinationUpload.** One document per submission. A FHIR upload keeps the whole Bundle in OriginalContent exactly as received, whatever its country or version, and adds ExtractedDoses, the normalised fields PRS needs, each with a back reference to the VaccinationRecordId it created. A PDF or image upload embeds only file metadata; the binary is in GridFS. A $jsonSchema validator requires OriginalContent for FHIR formats and File for PDF/IMAGE.
- **AuditLog.** A time-series collection (timeField Timestamp, metaField Actor). The actor's roles are embedded as a snapshot so the log shows what rights the user had at that moment, even if their roles change later. The application connects with a MongoDB role that can only insert and read, so entries cannot be edited or deleted.
- **Notification.** Each type carries a different Payload (stock nearby, limit warning, vaccination due, news). A TTL index deletes notifications after 90 days.
- **Announcement.** Public news and vaccination schedules with embedded attachment references and a text index for search.
- **Unstructured files (GridFS bucket PRSFiles).** PDF certificates and images are stored as 255 KB chunks in PRSFiles.chunks with metadata (PRSId, purpose) in PRSFiles.files. Large files never bloat the upload documents, and a SHA-256 hash prevents the same file being registered twice.

Embedding versus referencing

Data	Decision	Reason
OriginalContent (FHIR Bundle)	Embed	Owned by the upload, read together, small (well under 16 MB).
ExtractedDoses[]	Embed	Bounded list owned by the upload; shows exactly what was parsed.
File metadata	Embed	One per upload, always shown with it.
File content (PDF/image)	Reference (GridFS)	Binary, can be large, streamed on demand.
Person	Reference (PRSId)	Owned by SQL Server, shared by everything, updated independently.
VaccinationRecord	Reference (VaccinationRecordId)	Two-way link between the evidence and the normalised dose.
Actor in AuditLog	Embed snapshot	Must be historically accurate, not the user's current roles.
PublishedBy in Announcement	Embed snapshot	Display name read with every announcement.
Payload in Notification	Embed	Different shape per type; never queried on its own.

Table 3. Embedding and referencing decisions.

Example documents

The documents below were produced by importing the NHS sample (fhir_ingest.py) and a sample PDF (pdf_upload.py) into the implemented database. OriginalContent is shortened here; it holds the full bundle.

VaccinationUpload created from the NHS FHIR report:

```
{
  "_id": {
    "$oid": "6aca7b2b418a8b86ef76747f"
  },
  "PRSId": "PRS000000001",
  "UploadedByUserId": 1,
  "Format": "FHIR_JSON",
  "SourceCountry": "GB",
  "UploadedAt": {
    "$date": "2026-10-10T17:51:39.878Z"
  },
  "ProcessingStatus": "NeedsReview",
  "OriginalContent": {
    "resourceType": "Bundle",
    "type": "searchset",
    "total": 2,
    "entry": [
      "... 2 Immunization + 1 Patient resources, exactly as received ..."
    ]
  },
  "IdentityMatch": {
    "IdentifierTypeCode": "NHS_NUMBER",
    "MatchStatus": "Matched"
  },
  "ExtractedDoses": [
    {
      "SourceResourceId": "90640007.747583-127",
      "VaccineCode": "39114911000001105",
      "VaccineName": "COVID-19 Vaccine AstraZeneca (ChAdOx1 S ...",
      "OriginalDoseNumber": 1,
      "OccurrenceDateTime": {
        "$date": "2020-12-10T13:00:08.476Z"
      },
      "LotNumber": "R04X",
      "Manufacturer": "AstraZeneca Ltd",
      "AdministeredBy": {
        "OdsCode": "RX809",
        "Name": "TEST-SITE"
      },
      "CountryCode": "GB",
      "DoseNumber": 1,
      "VaccinationRecordId": 1
    },
    {
      "SourceResourceId": "90640007.747583-128",
      "VaccineCode": "39114911000001105",
      "VaccineName": "COVID-19 Vaccine AstraZeneca (ChAdOx1 S ...",
      "OriginalDoseNumber": 1,
      "OccurrenceDateTime": {
        "$date": "2020-12-31T13:00:08.476Z"
      },
      "LotNumber": "R04X",
      "Manufacturer": "AstraZeneca Ltd",
      "AdministeredBy": {
        "OdsCode": "RX809",
        "Name": "TEST-SITE"
      },
      "CountryCode": "GB",
      "DoseNumber": 2,
      "VaccinationRecordId": 2
    }
  ],
  "DataQualityFlags": [
    "DUPLICATE_DOSE_NUMBER"
  ]
}
```


VaccinationUpload for a PDF certificate (unstructured):


```
{
  "_id": {
    "$oid": "6aca7b2da813693868548b47"
  },
  "PRSId": "PRS000000001",
  "UploadedByUserId": 1,
```

```
  "Format": "PDF",
  "SourceCountry": "FR",
  "UploadedAt": {
    "$date": "2026-10-10T17:51:41.508Z"
  },
  "ProcessingStatus": "NeedsReview",
  "File": {
    "GridFsId": {
      "$oid": "6aca7b2da813693868548b45"
    },
    "FileName": "sample_certificate.pdf",
    "ContentType": "application/pdf",
    "SizeBytes": 1650,
    "Sha256": "1763d29ca1820ab4a8842c334444682d06fb2d8a94fcad74a5e32f2edeec084a"
  },
  "ExtractedDoses": [],
  "DataQualityFlags": [
    "UNSTRUCTURED_SOURCE"
  ]
}
```


AuditLog entry written by the import:


```
{
  "Timestamp": {
    "$date": "2026-10-10T17:51:39.878Z"
  },
  "Actor": {
    "Roles": [
      "Citizen"
    ],
    "UserId": 1,
    "Username": "ingest-service"
  },
  "IPAddress": "127.0.0.1",
  "Action": "CREATE",
  "_id": {
    "$oid": "6aca7b2b418a8b86ef767480"
  },
  "Details": {
    "Format": "FHIR_JSON",
    "Doses": 2,
    "Flags": [
      "DUPLICATE_DOSE_NUMBER"
    ]
  },
  "Target": {
    "Store": "MongoDB+SQL",
    "TargetEntity": "VaccinationUpload",
    "TargetRecordId": "6aca7b2b418a8b86ef76747f",
    "PRSId": "PRS000000001"
  },
  "Outcome": "Allowed"
}
```


Notification telling a person about nearby stock:


```
{
  "_id": {
    "$oid": "6aca7b2ba80bacb44ae9f4b2"
  },
  "PRSId": "PRS000000001",
  "NotificationType": "STOCK_NEARBY",
  "Channel": [
    "App"
  ],
  "CreatedAt": {
    "$date": "2026-10-10T17:51:39.427Z"
  },
  "ReadAt": null,
  "StoreId": 1,
  "Payload": {
    "ItemId": 1,
    "ItemName": "Face mask",
    "QuantityAvailable": 245,
```

```
    "DistanceKm": 0.64,
    "EligibleToday": false,
    "NextEligibleDay": "Thursday"
  }
}
```

Schema definition (extract)

MongoDB is schema-flexible, but PRS enforces the fields it depends on with a $jsonSchema validator. The extract below shows the rules for VaccinationUpload; the full set is in prs_mongodb_setup.js.

```
db.createCollection("VaccinationUpload", { validator: { $jsonSchema: {
  bsonType: "object",
  required: ["PRSId", "UploadedByUserId", "Format", "UploadedAt", "ProcessingStatus"],
  properties: {
    PRSId:            { bsonType: "string", pattern: "^PRS[0-9]{9}$" },
    UploadedByUserId: { bsonType: "int" },
    Format:           { enum: ["FHIR_JSON", "FHIR_XML", "PDF", "IMAGE"] },
    ProcessingStatus: { enum: ["Received", "Parsed", "NeedsReview", "Failed"] },
    OriginalContent:  { bsonType: ["object", "binData"] },   // binData when CSFLE-encrypted
    File: { bsonType: "object",
            required: ["GridFsId", "FileName", "ContentType", "SizeBytes", "Sha256"] },
    ExtractedDoses: { bsonType: "array", items: { bsonType: "object",
            required: ["VaccineCode", "DoseNumber", "OccurrenceDateTime"] } }
  },
  oneOf: [ { properties: { Format: { enum: ["FHIR_JSON", "FHIR_XML"] } }, required:
["OriginalContent"] },
           { properties: { Format: { enum: ["PDF", "IMAGE"] } },          required: ["File"]
} ]
}}, validationAction: "error" });
```

Handling messy imported data

The NHS sample shows why the original document must be kept and verified rather than trusted. Both Immunization resources say doseNumberPositiveInt = 1 and both are described as a first dose, although the second was given 21 days later; total = 2 counts only the Immunization entries. The import keeps the source value as OriginalDoseNumber, assigns DoseNumber by date order (1, then 2), adds the flag DUPLICATE_DOSE_NUMBER and sets ProcessingStatus to NeedsReview, so a verifying official decides. The Bundle also contains the NHS number and birth date, so OriginalContent is encrypted with Client-Side Field Level Encryption in production; the server stores only ciphertext.

Scalability

VaccinationUpload, AuditLog and Notification are the fastest-growing data in PRS. They can be sharded horizontally (hashed PRSId for uploads and notifications, Timestamp for the audit log) without affecting the transactional core, which stays compact in SQL Server and can use read replicas for the public locator.

3.4 Integrating the two models

The two stores form one design because they share identifiers, and those identifiers have the same names in every model:

Concept (Figure 1)	SQL Server (Figure 2)	MongoDB (Figure 3)
Person / PRSId	Person.PRSId (PK)	PRSId in VaccinationUpload, Notification; Target.PRSId in AuditLog

UserAccount / UserId	UserAccount.UserId (PK)	UploadedByUserId; Actor.UserId
Store / StoreId	Store.StoreId (PK)	Notification.StoreId
GovernmentOfficial / OfficialId	GovernmentOfficial.OfficialId (PK)	Announcement.PublishedBy.OfficialId
VaccinationUpload / UploadId	VaccinationRecord.UploadId (CHAR(24))	VaccinationUpload._id (ObjectId)
VaccinationRecord / VaccinationRecordId	VaccinationRecord.VaccinationRecordId (PK)	ExtractedDoses[].VaccinationRecordId

Table 4. Shared identifiers across the three models.

Because no single transaction spans both databases, writes follow a fixed order: the evidence is stored in MongoDB first, then the dose rows are written to SQL Server (idempotently), then the upload is updated with the new VaccinationRecordIds. If a step fails it can simply be repeated. A scheduled reconciliation job reports VaccinationRecords whose UploadId has no matching document and uploads left in Received state.

4 Conclusion

The PRS design separates a strict, transactional core from flexible, high-volume data. SQL Server holds identities, merchants, rules, purchases and normalised vaccination facts, where foreign keys, CHECK constraints and transactions guarantee that limits and schedules are applied consistently. MongoDB holds the data that does not fit tables well: FHIR reports whose structure varies, PDF and image certificates, the audit trail and notifications.

Key design choices:

- Encryption is chosen per attribute: deterministic for identifiers that must be searched, randomised for the DOB, with only the birth-year digit kept in clear so tills can enforce schedules without decrypting anything.
- Imported data is never trusted blindly: the original evidence is kept unchanged, normalised doses are linked to it, and quality problems such as the duplicated dose number in the NHS sample are flagged for an official.
- Rules are data. Limits, schedules, vaccination policies, roles and permissions are rows that officials can change during a pandemic without changing the schema.
- Consistency across models is built in: one definition generates every diagram and the DDL, and the same names link the two stores.

The main trade-off is that integrity between the two databases is maintained by the application and a reconciliation job rather than by foreign keys. This is accepted in exchange for storing evidence in its native form and scaling audit and upload data independently. Both models were implemented and tested, which shows the design supports the scenario's stories end to end.

References

- NHS England (2026) Immunisation History - FHIR API. Available at: https://digital.nhs.uk/developer/api-catalogue/immunisation-history-fhir

- HL7 International (2023) FHIR R4 Immunization resource. Available at: https://hl7.org/fhir/R4/immunization.html
- Microsoft (2025) Always Encrypted. Available at: https://learn.microsoft.com/sql/relational-databases/security/encryption/always-encrypted-database-engine
- MongoDB Inc. (2025) Schema Validation; GridFS; Time Series Collections; Client-Side Field Level Encryption. Available at: https://www.mongodb.com/docs/manual/

Appendix A Implementation files and test evidence

The models were implemented on Microsoft SQL Server 2022 and MongoDB 7.0 to check that they work. The files supplied with this report are:

File	Purpose
prs_sqlserver_schema.sql	Complete SQL Server DDL: 24 tables, constraints, indexes, seed data, procedures, view
prs_mongodb_setup.js	MongoDB collections, $jsonSchema validators, indexes and sample documents (mongosh)
fhir_ingest.py	Automatic import of a FHIR JSON report into both stores
pdf_upload.py	Stores a PDF/image certificate in GridFS and registers the upload
test_scenarios.sql	Purchase eligibility and constraint tests
fig*.svg / fig*.png	Full-resolution versions of every figure

Output of test_scenarios.sql (stories from section 2.3):

```
1. Masks on a Monday for someone born 1965 (allowed Thursday) -> expect rejected
Accepted Reason
-------- ------
0 Not an allowed day for this birth year
2. Five masks on Thursday -> expect accepted
Accepted Reason PurchaseId
-------- ------ ----------
1 Recorded 1
3. Three more masks the same week -> expect rejected (limit 7/week)
Accepted Reason
-------- ------
0 Limit 7 per Week exceeded (already bought 5)
4. Milk twice on the same day -> accepted then rejected (1/day)
Accepted Reason PurchaseId
-------- ------ ----------
1 Recorded 2
Accepted Reason
-------- ------
0 Limit 1 per Day exceeded (already bought 1)
5. Milk next day -> accepted
Accepted Reason PurchaseId
-------- ------ ----------
1 Recorded 3
6. Alerts raised
AlertId AlertType PRSId StoreId LimitId ScheduleId Status
------- --------- ----- ------- ------- ---------- ------
1 ScheduleBreach PRS000000001 1 NULL 7 Open
2 LimitBreach PRS000000001 1 2 NULL Open
3 LimitBreach PRS000000001 1 1 NULL Open
7. Nearest stores with face masks from Wolverhampton station
StoreId StoreName Postcode QuantityAvailable DistanceKm
------- --------- -------- ----------------- ----------
1 Corner Pharmacy - City Centre WV1 1AA 245 .64
8. Constraint checks (each should fail)
  rejected: The INSERT statement conflicted with the CHECK constraint "CK_UserAccount_1".
The conflict occurred in database "PRS", table "dbo.UserAccount".
  rejected: The INSERT statement conflicted with the CHECK constraint "CK_FamilyLink_1". The
```

```
conflict occurred in database "PRS", table "dbo.FamilyLink".
  rejected: The INSERT statement conflicted with the FOREIGN KEY constraint
"FK_StockLevel_StoreId". The conflict occurred in database "PRS", table "dbo.Store", column
'StoreId'.
```

Output of fhir_ingest.py on the NHS sample, run twice:

```
$ python3 fhir_ingest.py NHS-Immunization-FHIR-JSON.json 1
UploadId=6aca7b2b418a8b86ef76747f ProcessingStatus=NeedsReview
DataQualityFlags=['DUPLICATE_DOSE_NUMBER']
$ python3 fhir_ingest.py NHS-Immunization-FHIR-JSON.json 1   # same file again
UploadId=6aca7b2c2c25e634c0fca81f ProcessingStatus=NeedsReview
DataQualityFlags=['DUPLICATE_DOSE_NUMBER']
$ SELECT ... FROM VaccinationRecord
VaccinationRecordId PRSId DoseNumber OccurrenceDate SourceResourceId UploadId PolicyId
VerificationStatus
------------------- ----- ---------- -------------- ---------------- -------- --------
------------------
1 PRS000000001 1 2020-12-10 90640007.747583-127 6aca7b2b418a8b86ef76747f 1 Pending
2 PRS000000001 2 2020-12-31 90640007.747583-128 6aca7b2b418a8b86ef76747f 2 Pending
```